

Déployer Airflow sur son EC2

Procédure complète et fichiers corrigés pour Airflow 3.1 (Celery, Postgres, Redis, synchro S3) : secret dédié, rôle d'instance, déploiement automatique depuis GitHub, connexion privée au serveur MLflow.

[0 Vue d'ensemble](#)[1 Fichiers](#)[2 AWS](#)[3 GitHub](#)[4 Workflow](#)[5 Déployer](#)[6 Dépannage](#)

0 Ce qu'on déploie

Airflow 3.1 en mode Celery sur un EC2 dédié : api-server (interface web, port 8080), scheduler, dag-processor, worker, triggerer, Postgres et Redis, plus un conteneur qui synchronise les DAGs, les données et les logs avec `s3://nappecast/airflow/`. Les tâches envoient leurs runs au serveur MLflow par son IP privée.



Ressource	Valeur
Compte / région	844099234486 / eu-west-3
EC2	Amazon Linux 2023, t3.large (8 Go de RAM minimum), utilisateur SSH ec2-user
Security Group	airflow-sg : 22 et 8080 depuis vos IP
Rôle de l'instance	airflow-ec2-role : secret, pull ECR, S3 airflow/* et mlflow-artifacts/*
Secret	airflow/prod-latest (Secrets Manager)
Dépôt ECR	airflow, tags immuables
Rôle GitHub (OIDC)	github-actions-deploy, créé dans le document MLflow
Secrets GitHub	AIRFLOW_EC2_HOST, AIRFLOW_EC2_USER, AIRFLOW_EC2_SSH_KEY, AIRFLOW_SG_ID (environnement production)
MLflow	http://172.31.32.79:5000 (IP privée de l'EC2 MLflow)

Prérequis : le document MLflow est terminé (rôle github-actions-deploy et fournisseur OIDC créés). Sur votre poste : AWS CLI, jq et openssl.

CE QUI NE POUVAIT PAS FONCTIONNER DANS TES FICHIERS

- **deploy.sh cassé** : la ligne « BASH — /OPT/AIRFLOW/DEPLOY.SH » copiée depuis le PDF précède le `#!`, la dernière ligne est tronquée (`--env-file "`), le compte ECR est écrit `8440-9923-4486`, et le script fait un `chown` sur un `.env` qui n'existe plus.
- **Postgres et Airflow ne se connectaient pas** : la base était créée avec l'utilisateur par défaut `postgres` (pas de `POSTGRES_USER`), la base `airflow_db` et le mot de passe `xMys5224`, alors que la connexion d'Airflow visait l'utilisateur `airflow`, la base `airflow` et le mot de passe `motdepasse`.
- **Redis démarrait sans mot de passe** : `REDIS_PASSWORD` n'était pas dans le secret, et Compose remplaçait `$REDIS_PASSWORD` par une chaîne vide au chargement du fichier (vérifié avec `docker compose config`). L'URL du broker, elle, contenait un mot de passe.
- **`${S3_BUCKET:?...}` et `${AIRFLOW_UID}` non résolus** : Compose ne lit automatiquement qu'un fichier nommé `.env`. Avec `airflow.env`, il faut `--env-file airflow.env`.
- **Le script de synchronisation S3 perdait son horodatage** : Compose remplaçait aussi `$ts` par une chaîne vide. Il faut `$$ts`.
- **Le build échouait** : le `Dockerfile` copie `src/`, `configs/` et `docker/airflow/requirements.txt`, donc le contexte de build doit être la racine du dépôt, et `boto3==1.28.85` est incompatible avec `apache-airflow-providers-amazon`.

MOTS DE PASSE À RÉGÉNÉRER

Les valeurs de `aws_create_secret.sh` (clé Fernet, `xMys5224`, `motdepasse`) ont circulé en clair et le même mot de passe servait à trois usages. Le nouveau script `create_secret.sh` génère des valeurs aléatoires et distinctes, sans jamais les écrire dans un fichier du dépôt. Supprimez l'ancien : `git rm docker/airflow/aws_create_secret.sh`.

1 Les fichiers du dépôt

Tout est dans `docker/airflow/`. Le `Dockerfile` embarque aussi `src/` et `configs/` : un changement dans ces dossiers redéploie Airflow.

```
DOCKERFILE - docker/airflow/Dockerfile
FROM apache/airflow:3.1.3

USER root
RUN apt-get update && apt-get install -y --no-install-recommends \
    gosu \
    libpq-dev \
    && apt-get clean && rm -rf /var/lib/apt/lists/*
USER airflow

# Contexte de build = racine du dépôt (voir le workflow)
COPY docker/airflow/requirements.txt .
# On réinstalle la version d'Airflow de l'image pour que pip ne la change pas
RUN pip install --no-cache-dir "apache-airflow==3.1.3" -r requirements.txt

COPY src/ /opt/airflow/src/
COPY configs/ /opt/airflow/configs/

ENV PYTHONPATH=/opt/airflow
```

```

TEXT - docker/airflow/requirements.txt
# Providers Airflow (connexions Postgres & AWS/S3)
apache-airflow-providers-postgres
apache-airflow-providers-amazon

# boto3 n'est pas épinglé : sa version est imposée par apache-airflow-providers-amazon
pandas==2.3.3
plotly==6.9.0
requests==2.34.2
mlflow==3.14.0
pyyaml
python-dotenv
ccxt

```

`mlflow` complet est lourd. Si les DAGs ne font que journaliser des runs et des artefacts, `mlflow-skinny==3.14.0` suffit et évite des conflits de dépendances avec Airflow.

```

INI - docker/airflow/airflow.env
# Configuration NON secrète d'Airflow (versionnée dans Git).
# Les mots de passe et clés sont dans Secrets Manager (airflow/prod-latest).
AIRFLOW_UID=50000
POSTGRES_USER=airflow

# Synchronisation S3
S3_BUCKET=nappecast
DAGS_S3_PREFIX=airflow/dags
DATA_S3_PREFIX=airflow/data
LOGS_S3_PREFIX=airflow/logs
AWS_REGION=eu-west-3
AWS_DEFAULT_REGION=eu-west-3
SYNC_INTERVAL_SECONDS=60

# Serveur MLflow (IP privée de l'EC2 MLflow, joignable depuis l'EC2 Airflow)
MLFLOW_TRACKING_URI=http://172.31.32.79:5000
MLFLOW_EXPERIMENT_NAME=nappe-cast

```

Uniquement de la configuration non secrète. `MLFLOW_TRACKING_URI` pointe sur l'IP **privée** de l'EC2 MLflow : elle ne change pas quand l'instance est arrêtée, seulement si elle est recrée.

```

BASH - docker/airflow/create_secret.sh
#!/usr/bin/env bash
# Crée le secret Airflow avec des valeurs aléatoires. À lancer UNE SEULE FOIS, depuis votre poste.
# Ne contient aucun secret : peut être commité.
set -euo pipefail

SECRET_ID="airflow/prod-latest"
REGION="eu-west-3"

if aws secretsmanager describe-secret --secret-id "$SECRET_ID" --region "$REGION" >/dev/null 2>&1; then
    echo "Le secret $SECRET_ID existe déjà : rien n'est modifié (la clé Fernet ne doit pas changer)."
    exit 0
fi

PG=$(openssl rand -hex 24)
REDIS=$(openssl rand -hex 24)
ADMIN=$(openssl rand -hex 12)
FERNET=$(openssl rand -base64 32 | tr '+/' '-_')
API_KEY=$(openssl rand -hex 32)
JWT=$(openssl rand -hex 32)

umask 077
TMP=$(mktemp)
trap 'rm -f "$TMP"' EXIT

```

```
jq -n \
--arg pg "$PG" --arg redis "$REDIS" --arg admin "$ADMIN" \
--arg fernet "$FERNET" --arg api "$API_KEY" --arg jwt "$JWT" \
'{
  AIRFLOW__CORE__FERNET_KEY:      $fernet,
  AIRFLOW__API__SECRET_KEY:      $api,
  AIRFLOW__API__AUTH__JWT_SECRET: $jwt,
  POSTGRES_PASSWORD:            $pg,
  REDIS_PASSWORD:               $redis,
  AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: ("postgresql+psycopg2://airflow:" + $pg + "@postgres/airflow"),
  AIRFLOW__CELERY__RESULT_BACKEND: ("db+postgresql://airflow:" + $pg + "@postgres/airflow"),
  AIRFLOW__CELERY__BROKER_URL:      ("redis://" + $redis + "@redis:6379/0"),
  _AIRFLOW_WWW_USER_USERNAME:      "admin",
  _AIRFLOW_WWW_USER_PASSWORD:      $admin
}' > "$TMP"

aws secretsmanager create-secret --name "$SECRET_ID" --region "$REGION" \
--secret-string "file://$TMP" --query ARN --output text

echo "Mot de passe de l'interface (utilisateur admin) : $ADMIN"
echo "Notez-le maintenant, puis fermez ce terminal."
```

```
BASH - docker/airflow/deploy.sh

#!/usr/bin/env bash
# Exécuté sur l'EC2 Airflow par le workflow GitHub (sudo /opt/airflow/deploy.sh).
set -euo pipefail

APP_DIR=/opt/airflow
SECRET_ID="airflow/prod-latest"
REGION="eu-west-3"
ECR_REGISTRY="844099234486.dkr.ecr.eu-west-3.amazonaws.com"

cd "$APP_DIR"
for f in docker-compose.yml airflow.env; do
  [ -f "$f" ] || { echo "ERREUR : $APP_DIR/$f manquant"; exit 1; }
done

# 1. Secrets -> .env.runtime (lisible par root uniquement)
umask 077
aws secretsmanager get-secret-value --secret-id "$SECRET_ID" --region "$REGION" \
--query SecretString --output text > .secret.json
if ! jq -e 'type == "object"' .secret.json >/dev/null 2>&1; then
  rm -f .secret.json
  echo "ERREUR : le secret $SECRET_ID n'est pas un JSON valide"
  exit 1
fi
jq -r 'to_entries[] | "\(.key)=\(.value)"' .secret.json > .env.runtime
rm -f .secret.json
umask 022

# 2. Dossiers montés dans les conteneurs (utilisateur airflow = 50000)
mkdir -p dags logs plugins config data
chown 50000:0 dags logs plugins config data

# 3. Image + démarrage
aws ecr get-login-password --region "$REGION" \
| docker login --username AWS --password-stdin "$ECR_REGISTRY"

docker compose --env-file airflow.env pull
docker compose --env-file airflow.env up -d --remove-orphans
docker image prune -f
docker compose --env-file airflow.env ps
```

docker-compose.yml : on garde le fichier officiel d'Airflow que tu utilises, avec ces modifications (le fichier complet corrigé est fourni à part) :

```
YAML – docker/airflow/docker-compose.yml – modifications
# 1. Partout (x-airflow-common, postgres, redis, airflow-init)
env_file: [airflow.env, .env.runtime]          # au lieu de [.env, .env.runtime]

# 2. postgres : même nom de base que la connexion d'Airflow (l'utilisateur vient de airflow.env)
environment:
  POSTGRES_DB: airflow                        # au lieu de airflow_db

# 3. redis : $$ = variable lue par le conteneur, pas par Compose
command: ["sh", "-c", "exec redis-server --requirepass \"$$REDIS_PASSWORD\""]
healthcheck:
  test: ["CMD-SHELL", "redis-cli -a \"$$REDIS_PASSWORD\" --no-auth-warning ping | grep -q PONG"]

# 4. airflow-s3-sync : idem pour les variables du script
ts="$(date -u +%Y-%m-%dT%H:%M:%SZ)"
echo "[$$ts] dags : pull S3 -> EC2 OK"        # (même chose pour les 6 lignes echo)
```

LA RÈGLE DU \$ DANS DOCKER-COMPOSE.YML

- `${VAR}` ou `$VAR` : remplacé par **Compose** au chargement, depuis `--env-file airflow.env`. Sert pour `S3_BUCKET`, `AIRFLOW_UID`...
- `$$VAR` : laissé tel quel, puis lu par le **shell du conteneur**, qui voit les variables de `env_file`. Obligatoire pour les secrets (`REDIS_PASSWORD`) et les variables de script (`ts`).
- Pour vérifier : `docker compose --env-file airflow.env config` affiche un avertissement « variable is not set » pour chaque `$` mal échappé.

```
BASH – vérifications avant commit
chmod +x docker/airflow/deploy.sh docker/airflow/create_secret.sh
git rm docker/airflow/aws_create_secret.sh
git check-ignore -v docker/airflow/airflow.env    # ne doit RIEN afficher
cd docker/airflow && touch .env.runtime && docker compose --env-file airflow.env config -q; rm .env.runtime; cd -
```

2 AWS : créer l'infrastructure (une seule fois)

```
BASH
export AWS_DEFAULT_REGION=eu-west-3
export MYIP=$(curl -s https://checkip.amazonaws.com)
export MLFLOW_SG_ID=sg-06cdd4979e8928cf2
```

2.1 Dépôt ECR et secret

```
BASH
aws ecr create-repository --repository-name airflow --image-tag-mutability IMMUTABLE

./docker/airflow/create_secret.sh          # affiche le mot de passe admin : notez-le

# Contrôle : liste les clés, jamais les valeurs
aws secretsmanager get-secret-value --secret-id airflow/prod-latest \
  --query SecretString --output text | jq -e 'keys'
```

Le script ne touche jamais à un secret existant : la clé Fernet chiffre les connexions stockées par Airflow et ne doit plus changer une fois en production.

TON CAS : UN SECRET AIRFLOW/PROD-LATEST EXISTE DÉJÀ

Il a été créé par l'ancien script, avec des mots de passe incohérents et exposés. Supprime-le avant de lancer `create_secret.sh`. Si Postgres a déjà démarré sur l'instance avec l'ancien mot de passe, supprime aussi son volume : `POSTGRES_PASSWORD` ne s'applique qu'à la première initialisation.

```
BASH
aws secretsmanager delete-secret --secret-id airflow/prod-latest --force-delete-without-recovery
sleep 10 && ./docker/airflow/create_secret.sh

# Seulement si une instance Airflow a déjà tourné (efface la base Airflow)
ssh -i ~/.ssh/airflow-deploy.pem ec2-user@<EC2_HOST> \
  'cd /opt/airflow && sudo docker compose --env-file airflow.env down -v'
```

2.2 Rôle de l'instance : lire le secret, tirer l'image, synchroniser `airflow/*`, et écrire les artefacts MLflow. Ce dernier droit est nécessaire parce que le client MLflow envoie les artefacts **directement** dans S3, sans passer par le serveur.

```
BASH
cat > /tmp/ec2-trust.json <<'EOF'
{"Version": "2012-10-17", "Statement": [{"Effect": "Allow",
  "Principal": {"Service": "ec2.amazonaws.com"}, "Action": "sts:AssumeRole"}]}
EOF

cat > /tmp/airflow-ec2.json <<'EOF'
{
  "Version": "2012-10-17",
  "Statement": [
    {"Sid": "Secret", "Effect": "Allow", "Action": "secretsmanager:GetSecretValue",
      "Resource": "arn:aws:secretsmanager:eu-west-3:844099234486:secret:airflow/prod-latest-*"},
    {"Sid": "EcrAuth", "Effect": "Allow", "Action": "ecr:GetAuthorizationToken", "Resource": "*"},
    {"Sid": "EcrPull", "Effect": "Allow",
      "Action": ["ecr:BatchGetImage", "ecr:GetDownloadUrlForLayer"],
      "Resource": "arn:aws:ecr:eu-west-3:844099234486:repository/airflow"},
    {"Sid": "S3objets", "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"],
      "Resource": ["arn:aws:s3:::nappecast/airflow/*", "arn:aws:s3:::nappecast/mlflow-artifacts/*"]},
    {"Sid": "S3List", "Effect": "Allow", "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::nappecast",
      "Condition": {"StringLike": {"s3:prefix": ["airflow/*", "mlflow-artifacts/*"]}}}
  ]
}
EOF

aws iam create-role --role-name airflow-ec2-role --assume-role-policy-document file:///tmp/ec2-trust.json
aws iam put-role-policy --role-name airflow-ec2-role --policy-name airflow-ec2 \
  --policy-document file:///tmp/airflow-ec2.json
aws iam create-instance-profile --instance-profile-name airflow-ec2-role
aws iam add-role-to-instance-profile --instance-profile-name airflow-ec2-role --role-name airflow-ec2-role
```

Si vos DAGs lisent ou écrivent ailleurs dans le bucket (par exemple `nappecast/raw/`), ajoutez ces préfixes aux deux derniers blocs.

2.3 Clé SSH et Security Group

BASH

```
aws ec2 create-key-pair --key-name airflow-deploy --key-type ed25519 \
  --query KeyMaterial --output text > ~/.ssh/airflow-deploy.pem
chmod 400 ~/.ssh/airflow-deploy.pem

VPC_ID=$(aws ec2 describe-vpcs --filters Name=is-default,Values=true --query 'Vpcs[0].VpcId' --output text)
export SG_ID=$(aws ec2 create-security-group --group-name airflow-sg --description "Airflow" \
  --vpc-id $VPC_ID --query GroupId --output text)
aws ec2 authorize-security-group-ingress --group-id $SG_ID --protocol tcp --port 22 --cidr $MYIP/32
aws ec2 authorize-security-group-ingress --group-id $SG_ID --protocol tcp --port 8080 --cidr $MYIP/32
echo $SG_ID
```

2.4 Ouvrir MLflow à Airflow. Le Security Group de MLflow accepte le port 5000 depuis toute instance portant `airflow-sg`. Plus besoin de connaître l'IP d'Airflow :

BASH

```
aws ec2 authorize-security-group-ingress --group-id $MLFLOW_SG_ID \
  --protocol tcp --port 5000 --source-group $SG_ID
```

2.5 L'instance. Même User Data que pour MLflow. `HttpPutResponseHopLimit=2` permet aux conteneurs (worker, synchro S3) d'utiliser le rôle de l'instance.

BASH

```
cat > /tmp/userdata.sh <<'EOF'
#!/bin/bash
set -euo pipefail
dnf install -y docker jq
systemctl enable --now docker
mkdir -p /usr/local/lib/docker/cli-plugins
curl -fsSL "https://github.com/docker/compose/releases/latest/download/docker-compose-linux-$(uname -m)" \
  -o /usr/local/lib/docker/cli-plugins/docker-compose
chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
mkdir -p /opt/airflow
EOF

export INSTANCE_ID=$(aws ec2 run-instances \
  --image-id resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64 \
  --instance-type t3.large --key-name airflow-deploy --security-group-ids $SG_ID \
  --iam-instance-profile Name=airflow-ec2-role \
  --metadata-options HttpTokens=required,HttpPutResponseHopLimit=2 \
  --block-device-mappings 'DeviceName=/dev/xvda,Ebs={VolumeSize=30,VolumeType=gp3}' \
  --user-data file:///tmp/userdata.sh \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=airflow}]' \
  --query 'Instances[0].InstanceId' --output text)
aws ec2 wait instance-status-ok --instance-ids $INSTANCE_ID

ALLOC=$(aws ec2 allocate-address --domain vpc --query AllocationId --output text)
aws ec2 associate-address --instance-id $INSTANCE_ID --allocation-id $ALLOC
export EC2_HOST=$(aws ec2 describe-addresses --allocation-ids $ALLOC --query 'Addresses[0].PublicIp' --output text)
echo $EC2_HOST
```

BASH – vérification

```
ssh -i ~/.ssh/airflow-deploy.pem ec2-user@$EC2_HOST '
  docker compose version && jq --version &&
  aws secretsmanager get-secret-value --region eu-west-3 --secret-id airflow/prod-latest --query Name --output text &&
  curl -s --max-time 5 http://172.31.32.79:5000/health'
```

Les deux dernières lignes doivent afficher `airflow/prod-latest` puis `OK` : le rôle lit le secret, et MLflow est joignable.

3 GitHub : droits et secrets

3.1 Ajouter les droits Airflow au rôle `github-actions-deploy` (une policy de plus, à côté de celle de MLflow) :

```
BASH
cat > /tmp/gh-airflow.json <<EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {"Sid": "EcrAuth", "Effect": "Allow", "Action": "ecr:GetAuthorizationToken", "Resource": "*"},
    {"Sid": "EcrPushAirflow", "Effect": "Allow",
     "Action": ["ecr:BatchCheckLayerAvailability", "ecr:InitiateLayerUpload", "ecr:UploadLayerPart",
               "ecr:CompleteLayerUpload", "ecr:PutImage", "ecr:BatchGetImage", "ecr:DescribeImages"],
     "Resource": "arn:aws:ecr:eu-west-3:844099234486:repository/airflow"},
    {"Sid": "SshAirflow", "Effect": "Allow",
     "Action": ["ec2:AuthorizeSecurityGroupIngress", "ec2:RevokeSecurityGroupIngress"],
     "Resource": "arn:aws:ec2:eu-west-3:844099234486:security-group/$SG_ID"}
  ]
}
EOF
aws iam put-role-policy --role-name github-actions-deploy --policy-name github-actions-airflow \
--policy-document file:///tmp/gh-airflow.json
```

3.2 Les secrets GitHub, dans l'environnement `production` :

```
BASH
R=lauradmch/NappeCast-forecasting
gh secret set AIRFLOW_EC2_HOST --env production --repo $R --body "$EC2_HOST"
gh secret set AIRFLOW_EC2_USER --env production --repo $R --body "ec2-user"
gh secret set AIRFLOW_EC2_SSH_KEY --env production --repo $R < ~/.ssh/airflow-deploy.pem
gh secret set AIRFLOW_SG_ID --env production --repo $R --body "$SG_ID"
```

4 Le workflow

Même structure que celui de MLflow. Il n'y a que deux différences : il se déclenche aussi sur `src/**` et `configs/**`, et l'image est construite depuis la **racine** du dépôt (`context: .`).

```
YAML — .github/workflows/cd-airflow.yml
name: cd-airflow

on:
  push:
    branches: [master]
    paths:
      - 'docker/airflow/**'
      - 'src/**'
      - 'configs/**'
      - '.github/workflows/cd-airflow.yml'
  workflow_dispatch:

permissions:
  id-token: write
  contents: read

concurrency:
  group: cd-airflow
```



```

cancel-in-progress: false

env:
  SVC: airflow
  SVC_DIR: docker/airflow
  AWS_REGION: eu-west-3
  ECR_REGISTRY: 844099234486.dkr.ecr.eu-west-3.amazonaws.com

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4

      - name: Vérifier les fichiers et les secrets
        env:
          EC2_HOST: ${ secrets.AIRFLOW_EC2_HOST }
          EC2_USER: ${ secrets.AIRFLOW_EC2_USER }
          EC2_SSH_KEY: ${ secrets.AIRFLOW_EC2_SSH_KEY }
          SG_ID: ${ secrets.AIRFLOW_SG_ID }
        run: |
          ls -la "$SVC_DIR/"
          for f in Dockerfile docker-compose.yml "$SVC.env" deploy.sh; do
            test -f "$SVC_DIR/$f" || { echo "::error::$SVC_DIR/$f absent du commit"; exit 1; }
          done
          for v in EC2_HOST EC2_USER EC2_SSH_KEY SG_ID; do
            test -n "${!v}" || { echo "::error::secret AIRFLOW_$v non défini"; exit 1; }
          done

      - name: Auth AWS via OIDC
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::844099234486:role/github-actions-deploy
          aws-region: ${ env.AWS_REGION }

      - name: Login ECR
        uses: aws-actions/amazon-ecr-login@v2

      - name: L'image existe-t-elle déjà ?
        id: img
        run: |
          if aws ecr describe-images --repository-name "$SVC" \
            --image-ids imageTag="$GITHUB_SHA" >/dev/null 2>&1; then
            echo "exists=true" >> "$GITHUB_OUTPUT"
          else
            echo "exists=false" >> "$GITHUB_OUTPUT"
          fi

      - name: Set up Docker Buildx
        if: steps.img.outputs.exists == 'false'
        uses: docker/setup-buildx-action@v3

      - name: Build et push de l'image
        if: steps.img.outputs.exists == 'false'
        uses: docker/build-push-action@v6
        with:
          context: .
          file: docker/airflow/Dockerfile
          push: true
          tags: ${ env.ECR_REGISTRY }/${ env.SVC }:${ github.sha }
          cache-from: type=gha
          cache-to: type=gha,mode=max

      - name: Injecter le tag dans docker-compose.yml
        run: |
          sed -i "s|image: $ECR_REGISTRY/$SVC:[^ ]*|image: $ECR_REGISTRY/$SVC:$GITHUB_SHA|" "$SVC_DIR/docker-
compose.yml"

```

```

    grep -q "$SVC:$GITHUB_SHA" "$SVC_DIR/docker-compose.yml" \
    || { echo "::error::Tag non injecté dans docker-compose.yml"; exit 1; }

- name: IP publique du runner
  id: ip
  run: |
    IP=$(curl -sf https://checkip.amazonaws.com | tr -d '[:space:]')
    test -n "$IP" || { echo "::error::IP du runner introuvable"; exit 1; }
    echo "ip=$IP" >> "$GITHUB_OUTPUT"

- name: Ouvrir SSH pour ce runner
  id: open_ssh
  env:
    SG_ID: ${ secrets.AIRFLOW_SG_ID }
    RUNNER_IP: ${ steps.ip.outputs.ip }
  run: |
    aws ec2 authorize-security-group-ingress --group-id "$SG_ID" \
      --protocol tcp --port 22 --cidr "$RUNNER_IP/32"

- name: Préparer la clé SSH
  env:
    SSH_KEY: ${ secrets.AIRFLOW_EC2_SSH_KEY }
  run: |
    install -m 700 -d ~/.ssh
    printf '%s\n' "$SSH_KEY" > ~/.ssh/deploy_key
    chmod 600 ~/.ssh/deploy_key

- name: Copier les fichiers sur l'EC2
  env:
    HOST: ${ secrets.AIRFLOW_EC2_HOST }
    USER: ${ secrets.AIRFLOW_EC2_USER }
  run: |
    set -e
    OPTS="-i $HOME/.ssh/deploy_key -o StrictHostKeyChecking=accept-new -o ConnectTimeout=20"
    ssh $OPTS "$USER@$HOST" "rm -rf /tmp/$SVC-deploy && mkdir -p /tmp/$SVC-deploy"
    scp $OPTS "$SVC_DIR/docker-compose.yml" "$SVC_DIR/$SVC.env" "$SVC_DIR/deploy.sh" \
      "$USER@$HOST:/tmp/$SVC-deploy/"
    ssh $OPTS "$USER@$HOST" "ls -la /tmp/$SVC-deploy"

- name: Déploiement
  env:
    HOST: ${ secrets.AIRFLOW_EC2_HOST }
    USER: ${ secrets.AIRFLOW_EC2_USER }
  run: |
    ssh -i "$HOME/.ssh/deploy_key" -o StrictHostKeyChecking=accept-new -o ConnectTimeout=20 \
      "$USER@$HOST" "SVC=$SVC bash -s" <<'EOF'
    set -e
    D=/tmp/$SVC-deploy
    APP=/opt/$SVC
    sudo mkdir -p "$APP"
    sudo install -m 644 "$D/docker-compose.yml" "$D/$SVC.env" "$APP/"
    sudo install -m 755 "$D/deploy.sh" "$APP/deploy.sh"
    sudo "$APP/deploy.sh"
    rm -rf "$D"
    EOF

- name: Relever SSH
  if: always() && steps.open_ssh.outcome == 'success'
  env:
    SG_ID: ${ secrets.AIRFLOW_SG_ID }
    RUNNER_IP: ${ steps.ip.outputs.ip }
  run: |
    aws ec2 revoke-security-group-ingress --group-id "$SG_ID" \
      --protocol tcp --port 22 --cidr "$RUNNER_IP/32"

- name: Supprimer la clé SSH du runner
  if: always()
  run: rm -f ~/.ssh/deploy_key

```

5 Déployer et vérifier

BASH

```
git add docker/airflow .github/workflows/cd-airflow.yml
git commit -m "airflow: déploiement"
git push
```

Le premier démarrage prend quelques minutes : `airflow-init` crée la base, puis l'utilisateur `admin`. Ensuite :

BASH

```
ssh -i ~/.ssh/airflow-deploy.pem ec2-user@$EC2_HOST
cd /opt/airflow
sudo docker compose --env-file airflow.env ps          # tout en "healthy", airflow-init en
"exited (0)"
sudo docker compose --env-file airflow.env logs airflow-init | tail -20
sudo docker compose --env-file airflow.env logs --tail 5 airflow-s3-sync
sudo docker compose --env-file airflow.env exec airflow-worker sh -c 'curl -s "$MLFLOW_TRACKING_URI/health"'
```

Interface : `http://<EC2_HOST>:8080`, utilisateur `admin`, mot de passe affiché par `create_secret.sh`. Pour le relire :

BASH

```
aws secretsmanager get-secret-value --secret-id airflow/prod-latest \
--query SecretString --output text | jq -r '._AIRFLOW_WWW_USER_PASSWORD'
```

Les DAGs se déposent dans `s3://nappecast/airflow/dags/` : la synchro les récupère sur l'instance toutes les 60 secondes.

6 Dépannage

Symptôme	Cause	Solution
<code>password authentication failed for user "airflow"</code>	Le volume Postgres a été créé avec d'autres identifiants : <code>POSTGRES_PASSWORD</code> ne s'applique qu'à la première initialisation	Installation neuve : <code>docker compose down -v</code> (efface la base), puis redéployer
<code>NOAUTH</code> ou <code>invalid password</code> côté Redis/Celery	<code>\$REDIS_PASSWORD</code> non échappé, ou absent du secret	<code>\$\$REDIS_PASSWORD</code> dans le compose ; <code>jq keys</code> sur le secret
<code>Invalid auth token / Signature verification failed</code>	Composants avec des clés JWT différentes	<code>AIRFLOW_API_AUTH_JWT_SECRET</code> dans le secret (fourni par <code>create_secret.sh</code>)
<code>required variable S3_BUCKET is missing</code>	Compose lancé sans <code>--env-file airflow.env</code>	Toujours <code>docker compose --env-file airflow.env ...</code>
<code>ResolutionImpossible</code> pendant le build	Version épinglée incompatible avec Airflow	Retirer l'épinglage (comme <code>boto3</code>) ou suivre les contraintes Airflow
<code>COPY failed: ... src</code> pendant le build	Contexte de build = <code>docker/airflow</code>	<code>context: . + file: docker/airflow/Dockerfile</code>
Timeout vers <code>172.31.32.79:5000</code>	Règle 5000 absente du SG MLflow	Étape 2.4
<code>AccessDenied</code> S3 dans <code>airflow-s3-sync</code>	Préfixe non couvert, ou hop limit à 1	Étape 2.2 ; <code>modify-instance-metadata-options --http-put-response-hop-limit 2</code>

Symptôme	Cause	Solution
Conteneurs qui redémarrent, OOM	Instance trop petite	<code>t3.large</code> au minimum (8 Go)
<code>dial tcp ...:22: i/o timeout</code>	Instance arrêtée	<code>aws ec2 start-instances --instance-ids <id></code>

NE JAMAIS RÉGÉNÉRER LA CLÉ FERNET

`AIRFLOW__CORE__FERNET_KEY` chiffre les mots de passe des connexions enregistrées dans Airflow. Une fois en production, ne modifiez le secret qu'avec `put-secret-value`, en reprenant la même clé.

Compte `844099234486`, région `eu-west-3`, dépôt `lauradmch/NappeCast-forecasting`. À faire après le document « Déployer MLflow sur son EC2 ».